From Synthesized Controllers to a Playable Benchmark: The Design of a Human- and AI-Playable Game for Temporal Causal Reasoning

Abstract

We turn a formal account of temporal causation into a benchmark of temporal causal reasoning, grounded in TempoBench's temporal causal evaluation (TCE): given a synthesized finite-state system, an observed trace, and an output "o at time k," name the timestep-indexed inputs that caused it. TempoBench labels these with CORP, a tool in the Halpern–Pearl actual-causation tradition. We frame TCE through three causal objects over the same device—count-only *necessity*, *sufficiency*, and *actual causation* (necessity under a contingency)—and argue for actual causation as the core: we prove that on input-deterministic, input-total controllers every minimal count-only necessity cause is a *singleton*, and that none exists exactly when no single input flip destroys the effect (a verdict we call *no pivotal cell*). Count-only necessity therefore never returns a jointly-acting multi-cell cause, and returns nothing at all on overdetermined effects, where actual causation still has answers. Whether our actual-cause object equals the per-cell key TempoBench derives from CORP is stated as an explicit conjecture.

We operationalize the construction as TRACE (Temporal Reasoning And Causal Explanation): a synthesized controller frozen into a one-player switch-and-light device whose run flashes a target light. The player marks the switches that decided the flash (*Credit*) or plays the necessity (*Prevent*) and sufficiency (*Pin*) readings; answers are scored by a formal predicate that accepts any correct minimal cause, not a fixed string. The same interface is exposed to AI agents as a JSON protocol with a budgeted counterfactual probe and a verifiable reward. Humans and agents are graded on identical information within each of two tracks: a *white-box* track (the machine shown; the primary reasoning track, matching TempoBench's information setting) and a *black-box* track (the machine hidden; system identification plus causal reasoning, under an identifiability requirement).

TRACE is ARC-AGI-*inspired* in format—small, self-contained, procedurally generable, verifiably graded—though it tests active temporal credit assignment rather than ARC's static spatial induction. The paper closes with a concrete validation-and-build roadmap: auditing the released TempoBench artifacts against our assumptions (F1)–(F3), checking the CORP correspondence on a sample, profiling the grader, and running human and agent baselines. An appendix places the core at one corner of a lattice of richer variants, marking where automatic scoring remains tractable.

Contents

1 Introduction

1.1 The gap this benchmark targets

ARC-AGI *(Abstraction and Reasoning Corpus)* reopened a simple question: can we build a task that is easy for humans, hard for machines, and resistant to memorization, so that progress on it tracks reasoning rather than recall [3]? Its puzzles are small grid transformations specified by a handful of input/output examples; a solver must infer the rule and apply it to a held-out input. ARC-AGI deliberately measures *fluid* intelligence—efficient acquisition of a new skill from little data—rather than *crystallized* knowledge, and it is calibrated so that its tasks remain solvable by people while, at release, frontier models scored in the single digits.[1]

ARC-AGI is spatial and abstraction-driven. It says little about a second kind of reasoning that is central to programs, protocols, and agents: *temporal causal reasoning*—tracing, over a sequence of steps, which facts at or before an event were *necessary* for it. "Why did the grant fire at step 7?" "Which of the requests in the first five cycles actually mattered?" These are counterfactual questions about a temporal process, and they are exactly the kind of credit-assignment reasoning that an agent acting over time must do. TempoBench [11] introduced a formally grounded benchmark in this space, built from reactive-synthesis artifacts, with two tasks: *temporal trace evaluation* (TTE), deciding whether a finite trace is accepted by a supplied automaton, and *temporal causal evaluation* (TCE), asking which input facts caused a specified output at a specified timestep. TempoBench phrases TCE as the inputs "necessary" for the output but computes its answer key with CORP ("Causes for Omega-Regular Properties"), a cause synthesis tool in the Halpern–Pearl actual-causation tradition [8, 5, 10]; we take that gap between the prose (necessity) and the method (actual causation) seriously, because it determines what a "correct answer" is.

This paper does three things. First, it gives a self-contained, finite account of TCE in terms of three causal objects—necessity, sufficiency, and actual causation—argues that actual causation is the one that should match TempoBench's labels (a checkable conjecture), and makes "correct answer" a formal predicate rather than a convention. Second, it separates TempoBench's production pipeline (which labels causes with CORP) from Trace's own generator, isolating exactly where a ground-truth cause is computed and why it can be checked automatically. Third, it translates the construction into a concrete game, played and graded on identical information by humans and AI agents, and positions that game as a temporal-causal analog of ARC-AGI.

1.2 Why a formal cause makes a better benchmark

A benchmark is only as good as its answer key. ARC grids are hand-authored and graded by exact-output match; this is robust because the output is a single grid, but it scales by human effort and admits only one accepted answer. Temporal causal questions are different in two ways that the formal framework lets us exploit:

- **Ground truth is computable.** For a point effect "o at time k," validity of a candidate cause is a finite, decidable check over the supplied automaton—a reachability query for sufficiency,

[1] ARC-AGI-2 was released by the ARC Prize Foundation in March 2025; its technical report is [4] (submitted May 2025, revised January 2026). The report describes removing brute-force-solvable tasks and calibrating difficulty against timed human panels, with every retained task solved by multiple human testers, while pure language models scored near 0% and public reasoning systems reached only single-digit accuracy at release; current leaderboard figures may differ.

a closest-removal or contingency search for necessity and actual cause—and minimality is a search over a finite (if exponential) space of input-constraint sets. Instances and their answer keys can be *procedurally generated* from synthesized controllers, with difficulty tuned by observable features.

- **Correctness is set-valued, not string-valued.** Several incomparable minimal causes can exist for the same effect. A faithful grader therefore accepts *any* formally valid, subset-minimal cause, scored by a checker, rather than penalizing a correct answer for differing from one canonical string. This is a more honest test of reasoning than exact match, and it is only available because we can *decide* validity.

1.3 Contributions

(1) **A finite account on three causal objects.** We restate the minimal framework and the TempoBench frame $\mathfrak{F}_{\mathrm{TB}}$, and define *necessity*, *sufficiency*, and *actual causation* finitely over the point effect $X^k o$ (Section 2). We prove that count-only necessity collapses to singleton causes or the no-pivotal-cell verdict on input-deterministic, input-total controllers ((F2)–(F3); Proposition 2), which is why *actual causation*, not necessity, is the object aligned with TempoBench—an alignment we state as a checkable conjecture (Conjecture 1), not a theorem.

(2) **The pipeline as a benchmark generator.** We trace the path SYNTCOMP/TLSF → LTLsynt → HOA → HOAX trace → effect → cause, distinguishing TempoBench's CORP-labeled key from TRACE's own actual-cause generator, and analyzing its cost honestly (Section 3).

(3) **Trace: a human-playable game design.** The frozen controller as a Turing-Tumble-style switch-and-light device; three one-player modes (*Credit*/actual cause, *Prevent*/necessity, *Pin*/sufficiency); the budgeted counterfactual probe; predicate-based scoring at three granularities; difficulty knobs; and five fully worked puzzles (Section 4).

(4) **An information-parity AI-agent protocol.** A JSON observation/action interface matching the human game, with two tracks: a *white-box* primary track that shows the machine to human and agent alike (TempoBench's information setting), and a *black-box* track that hides it from both and adds a budgeted probe tool, subject to an identifiability requirement. Grading is a verifiable reward suitable for evaluation and RL, with a dated account of where the task is hard for current models (Section 5).

(5) **The ARC-AGI analogy, made precise.** A point-by-point comparison and a statement of what TRACE adds: decidable (predicate-checked) answers, procedural generation, and set-valued correctness (Section 6).

(6) **A validation-and-build roadmap.** A concrete, ordered list of the steps that take the design to a released benchmark—artifact audit, CORP cross-check, grader implementation and profiling, corpus statistics, identifiability enforcement, and human/agent baselines—each stated as a falsifiable TODO (Section 7).

(7) **A flexibility appendix.** Relaxing each TempoBench constraint axis yields a lattice of richer variants—bounded and liveness effects, output and strategy-structure interventions (the reverse parity game), profile-refined closeness, richer vocabularies, background assumptions, and the finite-to-infinite horizon step—with notes on which relaxations keep automatic scoring tractable and which do not (Appendix A).

Status of claims. *Established here:* the definitions and propositions of Section 2 (in particular the collapse of count-only necessity, Proposition 2), the grader predicates, and the game and protocol design. *Conditional:* every TempoBench-equivalence claim, which rests on three checkable artifact assumptions (F1)–(F3) and on Conjecture 1 (that our actual-cause object equals the per-cell key TempoBench derives from CORP). *Scheduled:* discharging (F1)–(F3) and Conjecture 1, implementing and profiling the generator and grader, and human/agent baselines are steps V1–V8 of the roadmap (Section 7).

2 The Framework, Restated for This Paper

This section is self-contained: it fixes the objects TRACE needs without assuming the companion documents. Readers familiar with the framework can skim to Section 2.4.

2.1 Systems, traces, and the supplied automaton

A reactive system has disjoint input and output propositions I and O ($I \cap O = \emptyset$). A complete valuation at one timestep is a set $\alpha \subseteq I \cup O$; the input part is $\alpha \cap I$ and the output part is $\alpha \cap O$. A *finite trace* of length $n+1$ is $\tau_{\leq n} = \alpha_0\alpha_1 \cdots \alpha_n$ with each $\alpha_t \subseteq I \cup O$. An atom p *holds at* t iff $p \in \alpha_t$; the literal $\neg p$ holds iff $p \notin \alpha_t$. We write $\models$ for a trace satisfying a temporal formula and read temporal operators over finite traces with LTL_f (finite-trace LTL) semantics [6]; the only operator we need is the bounded next, with $X^k o$ true on $\tau_{\leq n}$ iff $o \in \alpha_k$ (defined whenever $k \leq n$).

A synthesized controller is usually a Mealy machine $M = (S, s_0, I, O, \delta, \lambda)$, $\delta : S \times 2^I \to S$, $\lambda : S \times 2^I \to 2^O$. TempoBench, however, supplies the system as a finite-state *acceptor* rather than a transducer, serialized in the HOA (Hanoi Omega-Automata) format [1], a standard text format for ω-automata in which transitions are labelled by Boolean formulas over the atomic propositions.

Definition 1 (Supplied automaton and admissibility)**.** $A_{\mathrm{HOA}} = (Q, 2^{I \cup O}, \delta, q_0, F)$ *reads sequences of complete valuations. TempoBench's TTE task uses final-state acceptance (run defined and* $q_{n+1} \in F$*); for the* behavior-set *role we need here, a finite trace* $\beta = \beta_0 \cdots \beta_n$ *is* admissible *iff its run* $q_0 \xrightarrow{\beta_0} q_1 \cdots \xrightarrow{\beta_n} q_{n+1}$ *is merely* defined *(every step exists). That is, we read the synthesized controller as a safety/transducer object and do* not *use* F *to restrict behaviors (equivalently,* $F = Q$*);* $\mathcal{L}(A_{\mathrm{HOA}})$ *is then "the admissible behaviors of the system," not an effect monitor, and* $\mathcal{L}_{\leq n}(A_{\mathrm{HOA}})$ *is the admissible (run-defined) sequences of length* $n{+}1$*. We use "admissible" in this run-defined sense throughout (consistent with Section 3.2).*

2.2 The causal frame

The reverse-causal object is a *frame*, which fixes everything an explanatory question needs beyond the system itself:

$$\mathfrak{F} = (B_{\mathfrak{F}}, E_{\mathfrak{F}}, S_{\mathfrak{F}}, O_{\mathfrak{F}}, \mathcal{K}_{\mathfrak{F}}, \mathcal{I}_{\mathfrak{F}}, \preceq, \lhd).$$

Here $B_{\mathfrak{F}}$ is a background constraint (environment assumptions or held guarantees; true if none); $E_{\mathfrak{F}}$ is the effect to explain, possibly a selected conjunct $\bigwedge_{r \in S_{\mathfrak{F}}} E_r$ of a larger effect; $O_{\mathfrak{F}} \subseteq O$ the relevant outputs; $\mathcal{K}_{\mathfrak{F}}$ the candidate-cause space; $\mathcal{I}_{\mathfrak{F}}$ the intervention model; $\preceq = \{\preceq_\tau\}$ the similarity orders on traces; and $\lhd$ a preference order on candidates ($C' \lhd C$: C' is preferred). More simply: *M (or A_{HOA}) gives behavior; $\mathfrak{F}$ gives explanatory scope.* A cause is always reported *relative to* a declared frame.

The intervention model $\mathcal{I}_{\mathfrak{F}}$ fixes the admissible counterfactual set $\mathrm{Adm}^{\mathfrak{F}}_M(\tau)$. The default and only regime needed for TempoBench is *input-only*: counterfactuals stay genuine behaviors and

respect the background, $\mathrm{Adm}^{\mathfrak{F}}_M(\tau) = \mathcal{L}(M) \cap \mathcal{L}(B_{\mathfrak{F}})$ (with $\mathcal{L}(A_{\mathrm{HOA}})$ in place of $\mathcal{L}(M)$). Two further regimes—editing outputs, or re-opening the controller's strategic commitments—are the subject of Appendix A.

2.3 Closeness and valid causes

Counterfactual necessity needs a notion of "closest." Throughout, the unit of change is an *input cell*: a pair $(t, i) \in \{0, \ldots, n\} \times I$ recording the value of one input proposition at one timestep. Because outputs are determined by inputs (assumption (F2), Remark 1), a counterfactual is fixed by its input cells alone, and its outputs—including the effect—are whatever the machine then produces. We therefore measure distance by the *chosen* input changes only, never by their determined output consequences (otherwise flipping one early input, which the machine propagates to many later outputs, would be counted as a large change). The canonical order is *count-only*: for the actual trace τ, let

$$\mathrm{Diff}_\tau(\tau') = \big\{(t, i) \in \{0, \ldots, n\} \times I \;\big|\; [\, i \in \tau'(t)\,] \neq [\, i \in \tau(t)\,]\big\}, \qquad d(\tau, \tau') = |\mathrm{Diff}_\tau(\tau')|,$$

so fewer changed *input cells* is closer. Under this count-only order the closest removals form a *set* (all minimum-count witnesses), and validity quantifies over all of them. The closest admissible counterfactuals removing C are

$$\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C) = \min_{\preceq_\tau}\{\tau' \in \mathrm{Adm}^{\mathfrak{F}}_M(\tau) \mid \tau' \models \neg C\}.$$

Definition 2 (Valid cause)**.** *$C \in \mathcal{K}_{\mathfrak{F}}$ is a* valid cause *of $E_{\mathfrak{F}}$ on τ iff all four clauses hold:*

(i) ***Actuality:*** *$\tau \models C$ and $\tau \models E_{\mathfrak{F}}$;*

(ii) ***Non-vacuity:*** *$\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C) \neq \emptyset$ (some admissible trace removes C);*

(iii) ***Counterfactual dependence:*** *$\mathrm{Close}^{\mathfrak{F}}_{\tau,M}(\neg C) \cap \mathcal{L}(E_{\mathfrak{F}}) = \emptyset$ (the effect fails on every closest removal);*

(iv) ***Anti-circularity (syntactic):*** *C does not name the effect atom at the effect position—for a point effect $X^k o$, no literal of C mentions o at time k. A candidate may thus cause E* through *the machine but may not* restate *it.*

$\mathsf{Causes}_{M,\mathfrak{F},\tau}(E_{\mathfrak{F}})$ is the set of valid causes. A minimal *cause $C^\star$ is a $\lhd$-minimal element; when $\lhd$ is partial, several incomparable minimal causes may exist.*

Clause (ii) is what makes an answer non-trivial: a condition that no admissible counterfactual can violate (a tautology, or an invariant of the system under input-only intervention) is vacuous, not causal. Clause (iv) is a purely *syntactic* guard on the candidate. We deliberately do *not* test the semantic inclusion $\mathcal{L}(C) \subseteq \mathcal{L}(E_{\mathfrak{F}})$: in a deterministic system an input can entail the output *precisely because it causes it*, so a semantic test would wrongly reject genuine causes. Forbidding a candidate from mentioning the effect atom at the effect position is enough to bar a "cause" that merely re-asserts the effect.

2.4 The TempoBench frame

The TempoBench frame $\mathfrak{F}_{\mathrm{TB}}$ fixes a deliberately simple setting:

- **Effect class:** a single positive output occurrence at a fixed timestep, $\mathcal{E}_{\mathrm{TB}} = \{X^k o \mid o \in O_{\mathfrak{F}},\ 0 \leq k \leq n,\ o \in \alpha_k\}$. The selected effect is $E_{\mathfrak{F}} = X^k o$.
- **Background:** $B_{\mathfrak{F}} = \mathsf{true}$.
- **Intervention:** input-only; the automaton, outputs, transitions, and acceptance are not edited.

- **Candidate vocabulary:** timestep-indexed input literals over the causal window $0, \ldots, k$. Here a *literal* $\ell \in \mathrm{Lit}(I)$ is an input proposition or its negation, $\mathrm{Lit}(I) = \{i, \neg i \mid i \in I\}$, and a pair (t, ℓ) reads "literal ℓ is required at timestep t." A candidate is then a constraint set $\Gamma \subseteq \{0, \ldots, k\} \times \mathrm{Lit}(I)$, denoting the bounded formula $C_\Gamma = \bigwedge_{(t,\ell)\in\Gamma} X^t \ell$. There are at most $2|I|(k+1)$ literals, hence $|\mathcal{K}_{\mathfrak{F}_{\mathrm{TB}}}| \leq 2^{2|I|(k+1)}$ constraint sets—finite but exponential in the window. (A set containing both i and $\neg i$ at one timestep is contradictory and fails actuality; since exactly one literal per input cell holds on the fixed actual trace, the candidates that can pass actuality number at most $2^{|I|(k+1)}$.)
- **Closeness:** the count-only coarsening (fewer changed input cells is closer); Close is the full minimum-count set.
- **Preference:** subset minimality on Γ (not formula size or automaton size).

Finite valid causes. A counterfactual is itself a finite trace $\beta = \beta_0 \cdots \beta_n$ (a valuation sequence of the same length as $\tau_{\leq n}$, with each $\beta_t \subseteq I \cup O$ and β_k its valuation at the effect position); it is *admissible* when accepted by A_{HOA}, i.e. $\beta \in \mathrm{Adm}_A^{\leq k} = \mathcal{L}_{\leq n}(A_{\mathrm{HOA}})$. A *removal* for Γ is an admissible β that violates C_Γ (it changes at least one required input literal). With the change-counting order,

$$\mathrm{Close}^{\leq k}_{\tau_{\leq n},A}(\neg C_\Gamma) = \min_{\#\mathrm{changes}}\{\beta \in \mathrm{Adm}_A^{\leq k} \mid \beta \not\models C_\Gamma\},$$

$$\mathsf{Causes}^{\leq n}_{A,\mathfrak{F}_{\mathrm{TB}},k}(X^k o) = \{\Gamma \mid \tau_{\leq n} \models C_\Gamma,\ o \in \alpha_k,\ \mathrm{Close}^{\leq k}_{\tau_{\leq n},A}(\neg C_\Gamma) \neq \emptyset,$$
$$\forall \beta \in \mathrm{Close}^{\leq k}_{\tau_{\leq n},A}(\neg C_\Gamma).\ o \notin \beta_k\}.$$

A minimal cause $C^\star = C_{\Gamma^\star}$ uses any subset-minimal valid $\Gamma^\star$ (several may exist). Anti-circularity (clause (iv)) is automatic here: candidates are input literals and the effect is an output occurrence, so a candidate never names the effect atom at the effect position.

2.5 The reduction, and its two assumptions

Remark 1 (What A_{HOA} must be). The finite construction models the supplied automaton as the system's behavior set, which requires two assumptions, both checkable against the artifacts. **(F1) Behavior model:** $\mathcal{L}(A_{\mathrm{HOA}})$ is exactly the set of system executions, not a specification monitor A_φ whose language is the specification-satisfying traces. **(F2) Functionality:** the behavior set is functional in the inputs (input-deterministic), so each input sequence has at most one accepted $I \cup O$ completion—consistent with TempoBench's automata coming from reactive synthesis, and exactly what lets us speak of "the" output at each timestep. Both are mechanically checkable per corpus; auditing the released HOA artifacts against (F1)–(F3) is roadmap step V1 (Section 7). Until then the alignment is conditional: if (F1) fails because the supplied automata are specification monitors, the construction does not apply to those artifacts.

Proposition 1 (Finite specialization—a definitional correspondence)**.** *Assume (F1)–(F2). With* $\mathfrak{F}_{\mathrm{TB}}$ *as the input-only frame,* $B_{\mathfrak{F}} = \mathsf{true}$*, admissible set* $\mathcal{L}_{\leq n}(A_{\mathrm{HOA}})$*, the change-counting order, and subset minimality, and interpreting the general clauses of Definition 2 over finite traces with* LTL_f *semantics: for every* Γ*,* $\Gamma \in \mathsf{Causes}^{\leq n}_{A,\mathfrak{F}_{\mathrm{TB}},k}(X^k o)$ *iff* C_Γ *satisfies the four valid-cause clauses on* $\tau_{\leq n}$ *under* $\mathfrak{F}_{\mathrm{TB}}$*. The finite construction* is *the general definition restricted to a point effect; no infinitary acceptance for* A_{HOA} *is used.*

This is a definitional restatement, not a deep theorem: each clause refers only to positions $0, \ldots, k$, so it is a statement about length-$(k+1)$ prefixes under LTL_f. It pins down precisely what our count-only *necessity* object is; whether that object coincides with TempoBench's TCE labels

is a separate, semantic question taken up in Section 2.6, where count-only necessity turns out to be the wrong object for them.

Remark 2 (Validity is not monotone). Membership in $\mathsf{Causes}^{\leq n}$ is not monotone under $\subseteq$ on Γ: enlarging Γ can make a candidate valid (the cheapest removal must now flip more) or invalid (the closest removal flips an irrelevant added literal while keeping the output). So "delete literals while valid" need not reach a subset-minimal cause; computing a minimal cause needs genuine search. Each validity check is polynomial (finite reachability over A_{HOA} on the window); the search is the hard part. This non-monotonicity is also what makes the game non-trivial for a human or agent: there is no safe greedy shortcut.

2.6 Three causal objects, and what TempoBench computes

"Which inputs mattered for that flash?" has more than one precise reading, and the readings genuinely disagree. We make three explicit—*necessity*, *sufficiency*, and *actual causation*—show how they relate, and identify which one TempoBench's TCE labels compute. The game of Section 4 exposes all three as one-player tasks over the same device.

Why count-only necessity alone is not enough. The valid-cause object of Definition 2 is the strictest reading: an input cell matters only if, among the *fewest* possible changes, altering it already destroys the effect. Synthesized controllers are *typically* input-total, which we make explicit as a checkable assumption.

Assumption 1 (Input-totality, (F3))**.** *Every input sequence over $0, \ldots, n$ extends to an admissible (run-defined) trace of A_{HOA}; in particular, flipping any one input cell of $\tau_{\leq n}$ again yields an admissible trace. Like (F1)–(F2), this is mechanically checkable per corpus (roadmap step V1).*

Proposition 2 (Count-only necessity collapses to single pivotal inputs)**.** *Assume $\mathfrak{F}_{\mathrm{TB}}$ with count-only closeness, (F2), and (F3). Then every subset-minimal valid cause is a* singleton*: either some single input-cell flip falsifies $X^k o$—and each such cell $\{(t, \ell)\}$ is a minimal cause—or no single flip does, in which case $\mathsf{Causes}^{\leq n} = \emptyset$. We call the latter outcome the* no-pivotal-cell *verdict: no literal-conjunction valid cause exists.*

Proof. To violate $C_\Gamma = \bigwedge_{(t,\ell)\in\Gamma} X^t \ell$ it suffices to falsify one conjunct, and flipping that one input cell yields, by (F3), an admissible trace that is, by (F2), *unique*; so this is an admissible 1-change removal, the minimum change count is 1, and $\mathrm{Close}^{\leq k}(\neg C_\Gamma)$ is exactly the set of single flips of Γ's literals. If Γ is valid, every such flip falsifies $X^k o$; pick any $(t_0, \ell_0) \in \Gamma$ and note its single flip is the unique closest removal of $\{(t_0, \ell_0)\}$ and falsifies the effect, so $\{(t_0, \ell_0)\}$ is itself valid. Hence no valid Γ with $|\Gamma| \geq 2$ is subset-minimal. If instead no single flip falsifies $X^k o$, then no singleton is valid and (by the same argument) neither is any larger set, so $\mathsf{Causes}^{\leq n} = \emptyset$. □

Two clarifications sharpen the proposition. First, the collapse concerns the *shape of each minimal cause*, not the per-cell union: several distinct singleton pivots may coexist (Puzzle 4 has two), so the union of necessity causes can still span multiple cells. What count-only necessity can *never* return, under (F2)–(F3), is a jointly-acting multi-cell cause (a minimal cause of size ≥ 2). Second, the no-pivotal-cell verdict is itself ambiguous between two situations that the actual-cause object distinguishes: the effect may be *overdetermined*—no single flip suffices but actual causes exist under a contingency (Puzzles 3 and 5)—or it may be *input-unconditional on the window*—no combination of in-window input changes destroys it, in which case nothing is a cause under any of our readings, and the instance is discarded by the trivial-instance filter of Section 4.5. Count-only necessity thus

diverges from a Halpern–Pearl key exactly on overdetermined and jointly-overdetermining structure; how often that structure occurs in TempoBench's corpus is measured in roadmap step V4, and any nonempty key on an overdetermined instance already separates the two objects. This collapse of strict counterfactual dependence under overdetermination is exactly the phenomenon Halpern and Pearl introduced contingencies to handle [10]; our part is to make it precise for synthesized temporal controllers and to build on the reading that survives it.

What TempoBench's key generator computes: a form of actual causation. TempoBench computes its TCE answer key with *CORP* [8]: Algorithm 1 of the TempoBench paper invokes `corp` on the automaton, trace, and effect [11]. CORP synthesizes ω-regular causes for ω-regular effects, generalizing the *temporal causality* of Coenen et al. [5], a counterfactual, closest-trace account that its authors place in the actual-causation tradition of Halpern and Pearl [10]; CORP's causes are themselves ω-regular automata, which may express disjunctions and temporal patterns, not merely conjunctions of literals. The Halpern–Pearl tradition's defining move is the *contingency*: a cause must toggle the effect under *some* held-fixed setting of the other variables—not necessarily their actual setting—while, with the cause in place, the effect still obtains under that setting. Permitting a *non-actual* contingency is exactly what count-only closeness forbids, and it is what lets actual causation return answers without the collapse of Proposition 2. We now give the sufficiency and actual-cause objects finitely.

Definition 3 (Sufficient set / determining set)**.** *Fix A_{HOA}, $\tau_{\le n}$, and a point effect $X^k o$ with $o \in \alpha_k$. A set $\Gamma \subseteq \{0,\dots,k\} \times \mathrm{Lit}(I)$ with $\tau_{\le n} \models C_\Gamma$ is* sufficient *(it* pins *the effect) iff every admissible $\beta \in \mathcal{L}_{\le n}(A_{\mathrm{HOA}})$ with $\beta \models C_\Gamma$ has $o \in \beta_k$: fixing the inputs named by Γ to their actual values forces the output at k regardless of the other inputs. A* determining set *is a subset-minimal sufficient Γ. Sufficiency is decided by a finite non-emptiness check ("is there an admissible $\beta \models C_\Gamma$ with $o \notin \beta_k$?"); several incomparable determining sets may exist.*

Definition 4 (Actual cause; finite Halpern–Pearl, with non-actual contingencies)**.** *Fix A_{HOA}, $\tau_{\le n}$, and a point effect $X^k o$ with $o \in \alpha_k$. A* contingency *assigns (possibly non-actual) values w to a set $W \subseteq \{0,\dots,k\} \times I$ of in-window input cells. Under (F2)–(F3) each input assignment determines a unique admissible trace; write $\beta\langle W{=}w,\ \Gamma{=}v\rangle$ for the admissible trace with W set to w, the cells of Γ set to v, and every other in-window cell at its actual value. A nonempty $\Gamma \subseteq \{0,\dots,k\} \times \mathrm{Lit}(I)$ (its literals at their actual values, disjoint from W) is an* actual cause *of $X^k o$ iff:*

(AC1) actuality: *$\tau_{\le n} \models C_\Gamma$ and $o \in \alpha_k$;*

(AC2) contingency: *for some (W, w), both* (a, necessity) *$o \notin \beta\langle W{=}w,\ \Gamma{=}\overline{\mathrm{act}}\rangle_k$ (with W at w, flipping Γ off its actual values destroys the effect) and* (b, sufficiency) *for* every *subset $W' \subseteq W$, $o \in \beta\langle (W \setminus W'){=}w,\ W'{=}\mathrm{act},\ \Gamma{=}\mathrm{act}\rangle_k$ (with Γ in place the effect survives the contingency, and survives* robustly*: resetting any subset of the contingency cells to their actual values cannot destroy it; $W' = \emptyset$ recovers the plain case);*

(AC3) minimality: *no proper subset of Γ satisfies AC1–AC2.*

The sufficiency leg AC2(b) is what separates actual causation from bare counterfactual dependence: without it, a contingency that suppresses the effect on its own would spuriously license any candidate. On a singleton *candidate, AC2(a) with $W = \emptyset$ is exactly count-only counterfactual dependence (Definition 2); and since under (F2)–(F3) every count-only valid cause is a singleton (Proposition 2), the necessity* object *is the empty-contingency, singleton case of actual causation. (For a multi-cell candidate the two notions of "removal" differ—closest violation vs. flipping the whole candidate—so we make the special-case claim only for singletons.) The* actual-cause set *Γ^{ac} is the union of all (subset-minimal) actual causes—a* set of cells*, not itself a single cause; several incomparable actual causes may exist.*

Why this is Halpern–Pearl, and which Halpern–Pearl. The trellis is a structural causal model: the in-window input cells are the exogenous variables, the controller's deterministic transition-and-output map (under (F2)) is the system of structural equations, and o at k is the endogenous effect. AC1/AC2(a)/AC2(b)/AC3 instantiate the Halpern–Pearl conditions on this model in the *original/updated* variant [10], in which the contingency cells W may be held at *non-actual* values. This is not Halpern's later *modified* definition [9], which restricts W to its actual values; the choice matters exactly on overdetermined instances—under the modified definition Puzzle 3's cause would be the joint pair $\{(0, a), (0, b)\}$, not the two singletons we report. Conjecture 1 can therefore hold only if the contingency discipline of CORP's underlying definition of temporal causality [5] matches ours; establishing that is part of roadmap step V2. Independent of CORP, the variant choice is deliberate: a credit-assignment benchmark should attribute credit under symmetric overdetermination (Puzzle 3's "each switch is a cause" verdict matches the intuitive answer and is the one TempoBench-style per-cell keys can express), whereas the modified definition's joint answer cannot distinguish two independently sufficient switches from two switches that act only jointly (Puzzle 4)—a distinction the game is built to probe. Because every candidate cell here is exogenous, the usual HP subtlety of partitioning endogenous variables into held-fixed (W) and floating (Z) sets largely collapses, and the definition admits a plain restatement: Γ *is an actual cause iff there is a setting of the other input cells under which flipping* Γ *off its actual values destroys the effect while* Γ *at its actual values (robustly) preserves it.* We do *not* prove that this finite, literal-cell object coincides with CORP's ω-regular construction—that is Conjecture 1. Absent it, Definition 4 is best read as *trellis actual causation*: HP-style, but defined here.

How the three relate.

- **Count-only necessity** (Def. 2) is actual causation with the *empty* contingency on the *singleton* candidates it admits (Proposition 2)—strictest and cheapest, kept as a deliberately strict diagnostic (the *Prevent* mode).
- **Sufficiency** (Def. 3) is its *dual*, not a special case: a set that *forces* the effect over *all* admissible inputs (the *Pin* mode). This *global* sufficiency is the unconditional analog of AC2(b)'s *local* sufficiency (under a fixed contingency W)—related, but not the same object.
- **Actual causation** (Def. 4) combines a necessity leg (AC2(a)) and a local-sufficiency leg (AC2(b)) under a shared contingency, so an actual cause is, informally, a counterfactually necessary part of a locally sufficient set—a characterization standard in the Halpern–Pearl literature that we use as intuition, not a theorem. This is the core object TRACE grades against, and the one we *hypothesize* matches TempoBench's labels (Conjecture 1).

On Puzzle 3 below (effect fires if a *or* b is set, both set) count-only necessity reports no pivotal cell, whereas actual causation returns both $\{(0, a)\}$ and $\{(0, b)\}$ (hold the other off; each is then pivotal and, alone, sufficient). A caution about evidence: at the per-cell granularity TempoBench grades, *Credit* and *Pin* often *agree*—on every worked puzzle of Section 4.6 the actual-cause set equals the union of determining sets, and only *Prevent* differs by collapsing. Whether that per-cell equality holds in general on (F2)–(F3) instances is open: a proof would make the per-cell conjecture independent of the *Credit*-vs-*Pin* choice, and a counterexample would be exactly the instance that justifies *Credit* specifically. So the case for actual causation as the core is not that it out-predicts sufficiency cell-by-cell here; it is *definitional lineage*—CORP implements Halpern–Pearl temporal causality (necessity-under-contingency *and* sufficiency), not pure sufficiency synthesis, so neither half alone is its object. Where the readings genuinely diverge is in how cells group into minimal causes (the witness and all-causes granularities, Section 4.4), not in the per-cell union.

Conjecture 1 (Correspondence to CORP, at per-cell granularity)**.** *For TempoBench instances satisfying (F1)–(F3),* Γ^{ac} *(Definition 4) equals the timestep-indexed cell set of TempoBench's shipped TCE answer key—i.e. TempoBench's own per-cell rendering of CORP's output—for the same trace and effect. We state this as a* conjecture*. CORP's causes are ω-regular automata and may be disjunctive or temporally structured, whereas* Γ^{ac} *is a set of literal cells; rather than define our own flattening of an ω-regular cause to cells, we anchor the comparison on the per-cell key TempoBench itself derives and grades against, which makes the conjecture checkable against an observable artifact. What we establish unconditionally is structural:* Trace *computes a Halpern–Pearl actual-causation object over the same synthesized device. Confirming the conjecture—running TempoBench's key generation on a sample and comparing per-cell answers, with disagreements reported as failure cases—is roadmap step V2, the highest-priority item there.*

3 A Benchmark Generator for the TCE Setting

A benchmark needs three things: instances, answer keys, and a grader. We trace a pipeline that produces all three, from specification to certified cause. The front end—specification, synthesis, trace generation—is shared with TempoBench, which builds instances from SYNTCOMP/TLSF specifications synthesized by LTLsynt and drives them with the trace generator HOAX [11]. The back end is where Trace and TempoBench differ at the key step: TempoBench labels causes with CORP, whereas Trace computes the actual-cause object of Definition 4 by its own search; the two are conjectured to agree (Conjecture 1). Throughout, "the solver" is whoever answers the TCE question—a person, an agent, or our own ground-truth procedure.

Why an own grader rather than CORP as oracle. Three reasons, none of them a judgment on CORP. First, grading requires a *per-candidate predicate*: the checker must decide validity and minimality of any constraint set a player submits, whereas CORP *synthesizes* one ω-regular cause automaton per query—it is a generator, not a membership oracle at the literal-cell granularity Trace grades. Second, the finite-window, literal-cell objects of Section 2 feed the JSON protocol and the subset-minimality certificates directly, with no flattening step inside the grading loop. Third, keeping the key computation independent turns the CORP comparison (roadmap step V2) into a genuine cross-validation of both implementations rather than a self-check.

3.1 Production path

P1. Specification. Take a reactive-system specification from a standardized corpus. TempoBench draws on the SYNTCOMP benchmark suite [13], expressed in TLSF [12], which cleanly separates inputs, outputs, and their temporal behavior. A specification is an LTL (or TLSF) contract φ over $I \cup O$.

P2. Synthesis. Synthesize a controller realizing φ with a reactive-synthesis tool (ltlsynt from Spot [14]), producing an implementation in HOA format [1] that preserves the input/output partition. This is the artifact A_{HOA} of Definition 1; by construction it is an implementation, which is what makes assumptions (F1)–(F2) plausible (Remark 1).

P3. Trace generation. Drive A_{HOA} with an input sequence to produce a finite observed trace $\tau_{\leq n} = \alpha_0 \cdots \alpha_n$ over $I \cup O$ (this is what TempoBench's HOAX [7] does). Trace length n is a difficulty knob.

P4. Effect selection. Choose a target output occurrence: pick $o \in O_{\mathfrak{F}}$ and a timestep $k \leq n$ with $o \in \alpha_k$, giving the point effect $E_{\mathfrak{F}} = X^k o$. The causal window is $0, \ldots, k$; later timesteps are irrelevant to a point effect at k.

P5. Ground-truth cause. This is the step that is ours, not TempoBench's: where TempoBench calls CORP, TRACE computes the actual-cause object of Definition 4 for $X^k o$ under $\mathfrak{F}_{\mathrm{TB}}$ (the actual-cause set Γ^{ac} and its minimal witnesses; Section 3.2). Under Conjecture 1 the two keys coincide on instances satisfying (F1)–(F3); the key the grader certifies is TRACE's actual-cause object.

P6. Packaging. Emit the instance—A_{HOA}, the AP list, the trace $\tau_{\leq n}$, and the effect $X^k o$—together with the hidden key $\Gamma^\star$ (or, better, the validity oracle itself; see Section 3.2).

The solver is given P1's outputs through P4 (the AP list, $\tau_{\leq n}$, and $X^k o$, plus A_{HOA} in the white-box track); it is *not* given φ, the synthesis trace, or any internal structure beyond the automaton. The two TRACE tracks differ exactly here: the *white-box* track (the primary reasoning track) exposes A_{HOA}—the information setting that matches TempoBench's TCE, where the solver is handed the machine—while the *black-box* track withholds it and the solver recovers behavior through the probe tool (Section 5). In neither case is the solver asked to synthesize a controller, choose a trace or effect, or recover φ: it performs finite-horizon causal credit assignment over a provided execution. TempoBench's reported scores bear on the white-box (machine-given) setting, not the black-box one.

3.2 Computing and certifying the answer key

The whole benchmark rests on step P5, so we make it precise.

What the automaton means. We read A_{HOA} as an explicitly represented, transducer-style *behavior set*, not an effect monitor: a finite valuation sequence $\beta_{\leq n}$ is *admissible* iff it labels a run from q_0 whose every step is defined. Under (F2) each input sequence has exactly one admissible $I \cup O$ completion, so "the output at k" is well defined and a counterfactual is fully specified by its input cells on $0, \ldots, n$. We do not rely on a final-state acceptance condition: the prefixes we test are the admissible ones, and for a point effect $X^k o$ only the prefix $0, \ldots, k$ is relevant, because the output at k depends only on inputs through k (the causal, Mealy reading; later inputs cannot reach back).

Validity is a finite check. Fix a candidate Γ. Actuality is read directly off $\tau_{\leq n}$ ($\tau_{\leq n} \models C_\Gamma$ and $o \in \alpha_k$). For counterfactual dependence we (a) find the minimum number m of changed *input cells* that yields an admissible trace violating C_Γ, and (b) confirm that *every* such minimum-change admissible removal has $o \notin \beta_k$. Concretely, build the product of A_{HOA} with a small monitor that (i) enforces $\neg C_\Gamma$, (ii) tracks the number of input cells changed relative to $\tau_{\leq n}$, and (iii) records whether o holds at k; a breadth-first search over change count returns the closest removals and their effect status. Under (F2) the question "does an admissible completion with these inputs put o at k?" coincides with "simulate these inputs and read off o at k."

What a validity check costs. We state bounds rather than a blanket "polynomial." A caution first: HOA files store transition labels *symbolically*, as Boolean formulas over the atomic propositions [1], so the explicit transition graph must be obtained by a one-time expansion that is exponential in $|I|$ ($O(|Q| \cdot 2^{|I|})$ edges). All bounds below are stated against that *expanded*

graph: $|A_{\text{HOA}}|$ denotes the explicit size—states *and* input-labelled transitions, the latter already $O(|Q| \cdot 2^{|I|})$—and the expansion cost is paid once per instance, kept tractable by capping $|I|$ (a difficulty knob). Stated against the symbolic description instead, none of the polynomial claims below survive. Simulating one fixed input sequence is linear in $|A_{\text{HOA}}|$ and k. *Sufficiency* (Definition 3) is a reachability/non-emptiness query over the product—"can any admissible completion consistent with Γ avoid o at k?"—hence polynomial in $|A_{\text{HOA}}|$ and k. *Closest-removal necessity* (Definition 2) is a minimum-Hamming shortest-path search over the same explicit product (states $\times$ window position $\times$ change count); because the $2^{|I|}$ edge fan-out is *already* counted in $|A_{\text{HOA}}|$, this too is polynomial in $|A_{\text{HOA}}|$, k, and $|I|$, not separately exponential. The genuine blow-up is *actual causation* (Definition 4), which wraps the necessity check in an outer search over contingencies (W, w). With $N = |I|(k{+}1)$ binary in-window cells, a contingency is a partial assignment—each cell is unconstrained, held actual, or held flipped—so there are up to 3^N of them (for non-binary input encodings, more), and each carries up to $2^{|W|}$ AC2(b) subset-reset checks; per-candidate *Credit* checking is therefore exponential in $|I| \cdot k$. In short: sufficiency and necessity are polynomial in the explicit automaton; the core *Credit* mode is the one that is not, which is why generation leans on the difficulty knobs below. Measured grader runtimes over the targeted $(|I|, k)$ ranges are roadmap step V3.

Minimality requires search. By Remark 2 validity is not monotone, so a minimal cause is not reachable by greedy deletion. Computing a minimal cause—and, for the actual-cause object, the contingency W that witnesses it—is a genuine search over the (exponential) constraint space, by one of:

- **Exhaustive enumeration** by increasing $|\Gamma|$, returning the first valid set—feasible for small windows and input counts and the most direct way to certify subset-minimality;
- **Hitting-set / MaxSAT** encodings over the input-cell variables, with each validity check as a constraint or oracle call;
- **CEGIS** over the finite constraints, refining a candidate from counterfactual witnesses (each rejected competitor yields a trace that removes it yet preserves the effect).

Cost concentrates in the search, not in any single check (whose mode-dependent cost is as analyzed above). For benchmark *production* this is acceptable: the generator can afford the search, and difficulty knobs (small k, few inputs) keep it tractable.

Certify with the oracle, not the string. The cleanest answer key is not a single $\Gamma^\star$ string but the *validity oracle* $V : \Gamma \mapsto \{\text{valid}, \text{invalid}\}$ together with the minimality test. Because several incomparable subset-minimal causes can exist, shipping one canonical $\Gamma^\star$ and grading by exact match would mark correct reasoning wrong. Instead the grader, given a submitted Γ, checks: (1) Γ is valid for the active mode ($V(\Gamma)$ holds), and (2) Γ is subset-minimal (no $\Gamma' \subsetneq \Gamma$ is valid). The minimality test makes at most $2^{|\Gamma|}$ validity calls, each costing as analyzed above; because minimal causes are small this is fast in practice, but it is *not* a blanket polynomial-time claim, and the per-call cost is the mode-dependent cost just given. We return to this design point in Section 4.4.

3.3 What the pipeline guarantees, and what it costs

The pipeline yields instances whose answers are *decidable against a stated predicate*—no human labeling, and no dependence on a model's idea of the answer—with *tunable, observable* difficulty. ("Certified" here means certified against TRACE's actual-cause predicate; whether that predicate agrees with CORP's labels is Conjecture 1.) Its costs:

- **Generation cost** is dominated by the minimal-cause search, compounded for the actual-cause object by the outer contingency search; both are exponential in the worst case (Remark 2, and the cost analysis above), and the generator manages them with the difficulty knobs.
- **The (F1)–(F3) dependency** (Remark 1, Assumption 1) is a property of the upstream artifacts, not of the game; verifying it once for a corpus (roadmap step V1) discharges it for every instance drawn from that corpus.
- **Scope:** the core task is positive point effects under input-only intervention. Negative occurrences, bounded temporal patterns, liveness, and strategy-level causes are deliberately excluded here and reappear as game variants in Appendix A.

Difficulty features. TempoBench controls difficulty with five finite, observable features that carry over directly [11]: effect depth k, number of automaton states, transition count, the number of input cells in the true cause (*causal-input count*), and trace diversity (the number of distinct inputs exercised in the trace). TRACE adds two knobs of its own: trace length n (more distractor context) and the probe budget $\beta_{\max}$ (Section 4.3). A generator can target a difficulty by sampling specifications and traces that hit chosen ranges, and—as ARC-AGI-2 does [4]—filter out instances solvable by brute force or trivial heuristics (e.g. instances whose minimal cause is a single literal in the same column as the effect).

4 Trace: The Game

The design goal is to turn the frozen one-player object of Section 2 into something a person enjoys with no logic background, while keeping the answer a formal causal object that an agent can also produce and the checker of Section 3.2 can grade. We call the game TRACE (Temporal Reasoning And Causal Explanation); one instance is a *puzzle*, a curated set a *corpus*.

4.1 Running synthesis backwards into a board

Forward, synthesis turns a two-player parity game into a one-player reaction: it computes a winning strategy and bakes it into the machine, after which the system no longer chooses—only the environment does. TRACE is that frozen object made physical. Picture a marble-and-switch device in the mechanical spirit of *Turing Tumble* or *Manufactoria*—flow puzzles that are secretly finite-state machines and that non-experts, including children, play for fun. Those toys display the whole board, as does TRACE's white-box track; the black-box track *hides* the wiring, which makes it harder, since inferring the machine's behavior becomes part of the task. A token flows left to right through the device; at each step you set a *switch* (an input); the token's lane is the machine's *state*; and in some cells a *light* flashes (an output). In neither track does the player need symbolic logic: you set switches, press *run*, and watch the device react—with, in the white-box track, the wiring diagram on the table beside you.

A puzzle shows four things, none requiring symbolic logic to read:

B1. The device — a reactive box you interact with by running it; its internal wiring is fixed and hidden (formally the acceptor A_{HOA}, used only by the generator and grader).

B2. The switch panel — your inputs I at each step $0, \ldots, n$, each a toggle you can set.

B3. The light panel — the outputs O the device flashed on the observed run, step by step.

B4. The target — one highlighted flash: light o at step k. Steps after k are dimmed; they cannot have caused a flash at k, so play stays in the window $0, \ldots, k$.

Mathematically the device is a *trellis*: the automaton unrolled in time, one column of state-lanes per step, each switch setting an edge to the next column, and the observed run a single lit path that ends on a goal node—a cell that flashes the target light (Figure 1). The trellis is our *depiction* of the machine, used in the worked examples below. The *white-box* track shows the trellis itself; in the *black-box* track the player sees only the switch and light panels and the run—not the state-lanes—and recovers the device's behavior by experiment (Section 5).

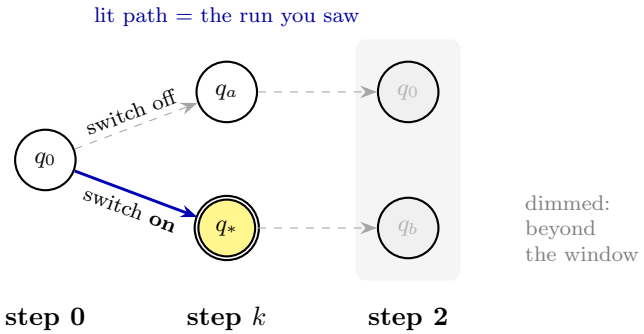


Figure 1: Anatomy of a TRACE board. Nodes are machine states; columns are timesteps; each edge is a switch setting. The thick blue *lit path* is the observed run, ending on the doubled *goal* node q_* that flashes the target light at step k. Dashed edges are untaken (counterfactual) routes; the shaded region past k is outside the causal window. *Prevent* reroutes the lit path off q_* with the fewest switch changes; *Pin* fixes the switches that keep every route on a goal node.

4.2 Three readings, three modes

The causal question—which earlier switch settings mattered for that flash—has the three precise readings of Section 2.6. TRACE exposes each as a one-player mode; the player-facing verbs are deliberately plain. The *core* mode is *Credit*, the actual-cause object we conjecture aligns with TempoBench (Conjecture 1); *Prevent* and *Pin* are its necessity and sufficiency halves, useful both as diagnostics and as the most immediately playable entry points.

Credit it (actual causation; the core). *Mark every switch that actually decided the flash.* A switch is a decider if there is *some* setting of the other switches—you may move them off their observed values—under which toggling it flips the flash while, with the switch in place, the flash still fires. You demonstrate this with a *pair* of runs under that setting (switch in place vs. toggled), the two arms of the contingency. A submission is either one minimal *actual cause* (Definition 4; the witness task) or the cell set Γ^{ac} (the per-cell union task), graded as in Section 4.4. This is the multi-switch object we *hypothesize* matches TempoBench's per-cell answers (Conjecture 1) and that escapes the collapse of Proposition 2.

Prevent it (count-only necessity; a strict diagnostic). *Find a single switch whose flip—by itself, everything else left as it was—stops the flash.* That pivotal switch is the cause; a submission is a singleton Γ, correct iff it is *valid* (Definition 2) and subset-minimal. By Proposition 2, on input-deterministic, input-total controllers ((F2)–(F3)) this is always either a single pivotal switch or the

verdict *no pivotal cell*—no single switch alone stops the flash. The verb matches the predicate: *Prevent* asks for a switch pivotal *on its own*, *not* for the smallest group of changes that would prevent the flash. Those differ—in Puzzle 3 two changes prevent the flash yet no single switch is pivotal, so the count-only answer is correctly "no pivotal cell"—which is exactly why bare necessity is too weak for TCE and why *Credit* is the core.

Pin it (sufficiency; the determining-set object). The dual: *lock the fewest switches at their settings so the target light flashes no matter how the other switches are set.* The locked set is what forces the flash; a submission is correct iff it is *sufficient* (Definition 3) and subset-minimal. *Pin* routinely yields multi-switch answers, and an actual cause is exactly a switch counterfactually necessary *within* some such determining set (Section 2.6).

All three are one-player: you only set or mark switches, and the "for some setting of the others" (*Credit*) and "no matter how the others are set" (*Pin*) are the grader's quantifiers—finite checks over the same frozen device—not a live opponent. A live adversary would change the question from "what caused *this* run" to "what can the controller *guarantee* against *every* environment"—a strategy-level object that needs the synthesis game rather than the device, kept as a separate variant in Appendix A, not the core.

Benchmark configurations (to prototype and compare).

Trace-Credit (core): the actual-cause object—multi-switch answers conjectured to match TempoBench's per-cell labels (Conjecture 1).

Trace-Prevent (diagnostic): count-only necessity only—single pivotal switches or the no-pivotal-cell verdict (Proposition 2); the cheapest, strictest mode.

Trace-Pin (diagnostic / dual): determining-set sufficiency only—the multi-switch "what forces the flash" question.

4.3 The try button: probe and rewind

Players act by experiment, not by reading the wiring. *Set* any switches in the window—to their observed values or to new ones—and press *run*; the device replays from the start, the light panel recomputes (deterministically, by (F2)), and you see whether the target flashes and how many switches you changed from the observed run. One press evaluates the active mode's inner question on *one* scenario:

- *Prevent*: did flipping this one switch (others left as observed) stop the flash?
- *Credit*: under this chosen setting of the other switches, does the candidate decide the flash? (A contingency test is a *pair* of presses—candidate in place vs. toggled.)
- *Pin*: with these switches locked and the rest set this way, does the flash survive?

A single press cannot settle *Pin*'s "for all other settings" or *Credit*'s "for some setting" by itself—those quantifiers are the *grader*'s, discharged by the finite check of Section 3.2; a press gives the player evidence about one scenario, from which they must generalize.

Presses are budgeted, and the budget is what forces *reasoning* over search. The space of settings over a window of depth k is $2^{\Theta(|I|\,k)}$, so a budget of only $O(k)$ presses cannot enumerate it: the player must form and test *compositional* hypotheses about how the device responds, which is the ability the benchmark means to measure. At $\beta_{\max} = 0$ the player must simulate the device in their head (pure deduction); a very large budget degrades the game to trial and error. Budget is therefore a difficulty knob (Section 4.5). A submission is read straight off the panel—marked cells (*Credit*), changed switch (*Prevent*), or locked switches (*Pin*)—so it is gradable identically for people and agents, with no free text.

4.4 Winning and scoring

A submission is graded by the formal checker of Section 3.2, against the *predicate* for the active mode, in one of three granularities.

Native (per mode). Graded against the active mode's predicate—an *actual cause* in *Credit* (Definition 4), a *valid* cause in *Prevent* (Definition 2), a *sufficient* set in *Pin* (Definition 3)—in one of three task variants, each with its own predicate:

- *Witness:* submit one set; full credit iff it is correct *and subset-minimal*. Minimality is essential—a non-minimal superset scores 0, so "spam every cell" fails.
- *All-causes:* submit a family; full credit iff it is exactly the set of *all* minimal causes for the mode.
- *Per-cell union:* submit the cell set Γ^{ac}, the union of all minimal causes, graded per cell. This is the granularity TempoBench reports; note the union need not itself be a single minimal cause, nor decompose into singleton causes.

The checker accepts *any* correct witness, not a fixed string. Witness or per-cell union is the recommended primary metric, depending on whether the corpus targets a single explanation or TempoBench-style per-cell scoring.

Timestep-level (TS). Against a shipped canonical set, a step scores only if its whole predicted switch set matches at that step. This mirrors TempoBench's F_1(TS) metric—the metric behind its headline TCE numbers [11]—and is useful for leaderboards.

AP-level. Precision/recall/F_1 per switch cell against the canonical actual-cause set, mirroring TempoBench's F_1(AP) metric [11]. A smooth signal for training, but, being per-cell, it can give partial credit to an answer that is not a correct minimal cause for the mode.

Why set-valued grading matters. Answers are genuinely non-unique: *Credit* and *Prevent* can have several incomparable causes (Puzzle 4), and *Pin* several determining sets (Puzzle 3). Grading the witness task against one stored string would mark a correct minimal answer wrong for merely differing from it; native grading checks the *predicate* instead—a facility ARC-style exact-match grading lacks, and one the formal backbone provides. TempoBench's F_1(AP) sidesteps this for the per-cell union task, but being per-cell it gives partial credit to non-minimal or partly-wrong answers; TRACE's native predicate grading is what we add. A canonical set (e.g. the lexicographically least) is still shipped for the TS and AP granularities, but the primary score is native.

Attempts. As in ARC-AGI, a puzzle allows a few submissions and is *solved* if any attempt earns full native credit, bounding lucky guessing while tolerating one slip.

4.5 Difficulty knobs

TRACE difficulty is set by observable, reportable features, so a corpus can be stratified and calibrated against human panels exactly as ARC-AGI-2 is.

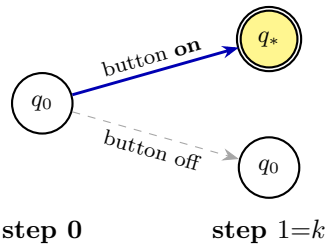
Knob	Effect on difficulty	Source
effect depth k	longer causal window, more credit assignment	TempoBench
# automaton states	richer internal dynamics to simulate	TempoBench
transition count	denser branching	TempoBench
causal-input count $\lvert\Gamma^\star\rvert$	larger true cause	TempoBench
trace diversity (# distinct inputs)	more candidate cells	TempoBench
trace length n	more distractor context	TRACE
probe budget $\beta_{\max}$	less brute-force checking	TRACE
mode (*Credit*/*Prevent*/*Pin*)	actual cause / necessity / sufficiency	TRACE
closeness order	count-only vs. profile (App. A)	framework

As in ARC-AGI-2, the generator should discard instances solvable without reasoning—e.g. those whose target is unconditional ($\Gamma^\star = \emptyset$), or whose cause is a single literal in the same column as the effect—and calibrate the retained set so that humans reliably solve it while the cheapest brute-force probe strategy does not.

4.6 Five worked puzzles

We trace the evaluation of five example puzzles. Each device is described in plain terms (its formal transition function stays under the hood) and drawn as a trellis in the style of Figure 1: the blue lit path is the observed run, the doubled yellow q_* is the target flash, and dashed edges are untaken routes. Closeness counts changed input cells, and for each puzzle we report all three readings—*Credit* (actual cause, the core), *Prevent* (count-only necessity), and *Pin* (sufficiency); each reported answer is the output of the finite checker of Section 3.2. Puzzles 1–2 have the three readings agree; Puzzles 3–5 show them diverge—the reason the core is actual causation, not bare necessity—with Puzzle 5 a genuinely multi-step (k=2) case.

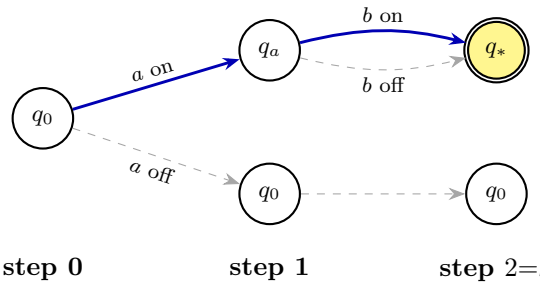
Puzzle 1 (one pivotal switch; tutorial). *Device:* a button and a lamp; the lamp flashes one step after the button is pressed. Window $\{0, 1\}$, target = lamp at step 1.



Lit blue path = observed run; doubled yellow q_* = the target flash; dashed = the *Prevent* reroute. One change (button off) leaves the goal.

Credit: with everything else fixed, toggling the button at step 0 flips the flash, so $\{(0, \text{button})\}$ is the lone actual cause. *Prevent:* releasing the button at step 0 (one change) stops the flash; pressing at step 1 does nothing—same pivotal switch. *Pin:* locking the button on at step 0 forces the flash regardless of step 1; same singleton. All three readings agree.

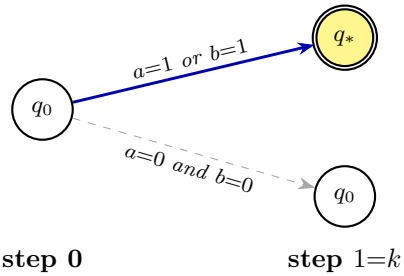
Puzzle 2 (delayed cause with a distractor). *Device:* two switches a, b and a lamp *done*; the lamp flashes exactly two steps after a is set, and b does nothing. Window $\{0, 1, 2\}$, target = *done* at step 2.



a at step 0 is pivotal (the a-off route never reaches q_*); b is a distractor—*both* b edges from q_a land on the flash.

Credit/Prevent/Pin: only a at step 0 matters—toggling it (under any setting of b) flips the flash, it alone prevents, and it alone forces—so all three readings give $\{(0, a)\}$. This is genuine delayed credit assignment (k=2) on which the readings still coincide; the traps are the two-step *delay* and the present-but-irrelevant b, which bait a recency or same-step guess.

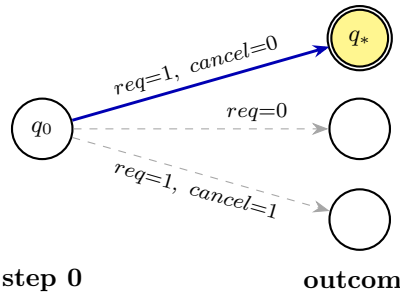
Puzzle 3 (overdetermination: the modes diverge). *Device:* switches a, b and a lamp *fire*; the lamp flashes one step after *either* switch is set. Both are set at step 0. Window $\{0, 1\}$, target = *fire* at step 1.



Only setting *both* switches off (two changes) escapes the flash, so no single change prevents it—*no pivotal cell* for *Prevent*. *Credit*: each switch is pivotal once the other is held off, so each is an actual cause. *Pin*: a=1 alone, or b=1 alone, already forces it (two determining sets).

Prevent: clearing a leaves b, which still fires; clearing b is symmetric; only the two-change removal escapes, while every one-change removal preserves the flash. So no single switch is pivotal and $\mathsf{Causes}^{\leq n} = \emptyset$: count-only necessity reports *no pivotal cell*, and since actual causes exist (next), the effect is genuinely overdetermined. *Credit:* hold b off (a contingency) and a becomes pivotal; hold a off and b becomes pivotal; so *both* $\{(0, a)\}$ and $\{(0, b)\}$ are actual causes—the multi-cell shape a Halpern–Pearl key can report, recovered exactly where *Prevent* saw nothing. *Pin:* locking a on forces the flash no matter what b does (and symmetrically), so there are *two* determining sets $\{(0, a)\}$ and $\{(0, b)\}$. The one instance reads as "no single switch was necessary" (*Prevent*), "each switch is a cause" (*Credit*), and "either alone suffices" (*Pin*)—the collapse of bare necessity, and why the core is actual causation.

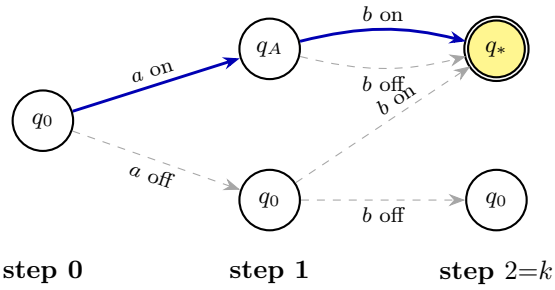
Puzzle 4 (a guard: the modes diverge the other way). *Device:* a request switch *req*, a cancel switch *cancel*, and a lamp *grant*; the lamp flashes at a step exactly when *req* is set *and* *cancel* is not, at that same step. At step 0, *req* is set and *cancel* is not. Window $\{0\}$, target = *grant* at step 0.



Only $req{=}1, cancel{=}0$ flashes *grant* (q_*). Flipping *either* switch escapes—two pivotal switches (*Prevent*); *Pin* must lock *both* to stay on q_*.

Prevent: clearing *req* stops the flash, and so does setting *cancel*; each single change works, so there are *two* pivotal switches $\{(0, req)\}$ and $\{(0, \neg cancel)\}$. *Credit:* each is pivotal under the actual setting of the other, so both are actual causes—necessity and actual cause here return the guard's parts *separately*. *Pin:* neither alone suffices (with *req* locked on, a set *cancel* still kills the flash), but locking *both* forces it, so the single determining set is the pair $\{(0, req), (0, \neg cancel)\}$—sufficiency returns the parts *together*. This is why a string-match grader is wrong in every mode, and why a corpus must declare which question it asks (Section 4.4, Appendix A). *A caveat:* with $k{=}0$ the cause sits in the same column as the effect, so an evaluation corpus would *discard* this instance under the trivial-instance filter of Section 4.5; we keep it here only because it isolates the necessity/sufficiency divergence in the smallest possible device. Puzzle 5 shows the same divergence at genuine temporal depth.

Puzzle 5 (delayed overdetermination; a genuinely multi-step divergence). *Device:* switch a (read at step 0) and switch b (read at step 1) and a lamp *fire*; *fire* flashes at step 2 if a was set at step 0 *or* b at step 1. Observed: a set at 0, b set at 1, *fire* at 2. Window $\{0, 1, 2\}$, target = *fire* at step 2.



From q_A (a already on) *both* b-edges reach the flash; from the a-off q_0, only b on reaches it. Each cause sits at a *different* timestep.

Prevent: clearing a at step 0 alone still fires (b at 1 remains), and clearing b at step 1 alone still fires (a at 0 remains); no single change prevents, so $\mathsf{Causes}^{\leq n} = \emptyset$—*no pivotal cell*, with the effect again genuinely overdetermined, exactly as in Puzzle 3 but now across two timesteps. *Credit:* hold b at 1 off and a at 0 becomes pivotal; hold a at 0 off and b at 1 becomes pivotal; so both $\{(0, a)\}$ and $\{(1, b)\}$ are actual causes—a multi-cell answer spread *across time*, recovered where *Prevent* saw nothing. *Pin:* a at 0 alone forces the flash, and b at 1 alone forces it, giving two determining sets $\{(0, a)\}$ and $\{(1, b)\}$. This is the case the benchmark is built for—credit assignment across a delay, where count-only necessity collapses and the actual-cause core does not.

5 Playing as an AI Agent

The same puzzle is exposed to an AI agent through a text protocol that mirrors the human device one-to-one. Parity is the point: within each track, the human and the agent receive *the same information* and the same probe tool, take the same actions, and are graded by the same checker. Nothing in the agent interface reveals the answer key or φ.

5.1 Observation: two tracks

White-box (primary). The primary track ships the device explicitly to *both* players—the agent gets the transition table or the raw HOA text TempoBench uses, the human sees the trellis of Figure 1. This is the information setting of TempoBench's TCE, and it isolates simulation-plus-counterfactual reasoning with no identification burden. One caveat governs its interpretation: an agent permitted to execute code can mechanize the grader's reachability search, so white-box agent evaluations must declare the tool regime—tool-less runs measure reasoning, code-equipped runs serve as a solver ceiling.

Black-box (discovery track). The second track withholds the machine from human and agent alike; both see only the switch and light panels of the observed run, the target, the mode, and the budgets, and learn how the device behaves only through the probe tool below. This defeats checker mechanization and adds system identification to the task—deliberately, but it changes what a score means: an error may reflect underdetermination rather than bad reasoning if two devices consistent with everything observed disagree on the answer. The track therefore carries an *identifiability requirement*: an instance is admitted only if every device consistent with the observed run and with the information obtainable within the probe budget yields the same native answer. Enforcing this in the generator is roadmap step V5; black-box scores are reported as measuring identification-plus-reasoning jointly.

The agent's observation (machine field present in white-box, absent in black-box):

```
{
"switches":  ["a","b"], "lights":  ["fire"],
"run":  [ {"t":0,"a":1,"b":1,"fire":0}, {"t":1,"a":0,"b":0,"fire":1} ],
"target":  { "light":"fire", "step":1 },
"mode":  "credit", "window":  [0,1], "probe_budget":  8, "max_attempts":  2
}
```

The `mode` field selects *credit*, *prevent*, or *pin*; the agent is told the rules, the scoring granularity, and the budgets.

A modality caveat. The human reads a moving board; the agent reads JSON. This is a difference in *presentation*, not in information, but spatial-versus-textual encoding could itself shift the measured human–agent gap, so a careful study should control for it—e.g. by also giving humans the JSON and agents a rendered board.

5.2 Action and answer

The agent answers with a grouped, by-step constraint set Γ, where each cell is a (timestep, variable : value) entry—so a negative-literal cause such as $(0, \neg \mathit{cancel})$ is simply `"cancel":0`, and Puzzle 4's guard encodes cleanly:

```
credit:  { "cause":  { "0":  {"req":1,"cancel":0} } }
prevent:  { "pivot":  { "0":  {"req":1} }, "no_pivot":  false }
pin:     { "lock":  { "0":  {"req":1,"cancel":0} } }
```

In *credit* the entries are one minimal actual cause or the union Γ^{ac}; in *pin*, the switches to lock. In *prevent*, `pivot` names the pivotal cell *at its actual value*—the cell whose flip (to its negation; inputs are binary) stops the flash—and the boolean `no_pivot` lets the agent give Puzzle 3's correct no-pivotal-cell verdict instead. In every mode the cells are stated at actual values; the grader supplies the flips. Grouping by step makes AP- and TS-level scoring direct.

5.3 The probe tool

To match the human "run" button, the agent may call a *probe* tool:

```
probe({ "0":  {"a":0} })  →  { "lights":  [...recomputed...], "target_flashes":
false, "switches_changed":  1 }
```

The tool is the programmatic equivalent of the human run button of Section 4.3, budgeted identically: it sets the named switch cells, re-simulates deterministically, and returns the recomputed lights, the target status, and the change count. A *credit* contingency test is a *pair* of probes under one setting of the other cells—candidate in place vs. toggled—costing two against the budget and exhibiting both arms (AC2(a), AC2(b)) of Definition 4. Two evaluation regimes follow:

- **Tool-use regime** ($\beta_{\max} > 0$): the agent probes to discover how the device responds and then reasons—guided experimentation, the temporal analog of running code to test a hypothesis. In the black-box track this is the *only* way to learn the machine, so $\beta_{\max} > 0$ is required there.
- **Pure-reasoning regime** ($\beta_{\max} = 0$): with the machine shown (white-box track), the agent must *simulate* it over the window and reason about the causal question with no execution. This isolates internal world-modeling and is the harder, more ARC-like setting. ($\beta_{\max} = 0$ with the machine hidden is excluded: one run does not fix the counterfactuals.)

5.4 Verifiable reward

Because grading is the formal checker, the reward is computed without a human or a reference model: full reward iff the submission is correct for the active mode (native), or graded partial credit (AP/TS modes). This makes TRACE a *verifiable-reward* environment suitable both for evaluation and for reinforcement learning, with no reward-model drift. The per-call cost is the mode-dependent cost of Section 3.2—polynomial in the explicit automaton and k for sufficiency and necessity, exponential in $|I| \cdot k$ for the core *Credit* mode (the contingency search)—and the minimality test adds $2^{|\Gamma|}$ calls, small because minimal causes are small. The benchmark targets small-window instances on which this is designed to be tractable at training scale, with *Credit* the costliest mode; measured grader throughput over the targeted $(|I|, k)$ ranges is roadmap step V3.

5.5 Why the task is hard for current models—and where it is not

The task is designed to resist the failure modes that inflate scores on text benchmarks. No natural-language world knowledge applies: each device is freshly generated, so there is no fact to retrieve—though instances are drawn from a fixed SYNTCOMP-derived distribution whose controller families (arbiters, AMBA-style protocols) have structure a model could in principle learn, a distributional prior the evaluation should measure rather than assume away. Solving a puzzle requires (i) *learning*

or simulating a finite-state machine over a window—multi-step state tracking that language models do unreliably as depth grows, and which in the black-box track must first be recovered from probes—and (ii) *counterfactual* reasoning that is provably non-monotone (Remark 2). We are careful about what non-monotonicity buys: it shows that no *exact* greedy or single-pass heuristic can be correct in general (Puzzles 2, 3, and 5), but it does *not* by itself show that *approximate* shortcuts score low, so we treat "memorization-proof" as a design goal to confirm empirically, not a theorem.

The closest existing evidence is TempoBench's own TCE results: aggregated across the evaluated models, timestep-level F_1 is 65.6% on TCE-normal but only 7.5% on TCE-hard [11]—models clearly "understand the task" yet degrade sharply as the device grows. So the ARC-like profile (near-floor machine scores against high human solve rates) holds at the *hard* stratum, not uniformly; on easy instances the models evaluated in the October 2025 TempoBench paper already do well. Two caveats temper even that: those are F_1 scores at the timestep granularity (TempoBench also reports a per-cell AP-level F_1, which is more permissive), and both give partial credit across the trace relative to TRACE's exact-answer native predicate; and the numbers predate any TRACE-specific calibration. We therefore position TRACE as targeting the hard stratum by construction (large k, more states, tight or zero probe budget) and leave the human–agent gap to roadmap steps V6–V7.

6 The ARC-AGI Analogy, Made Precise

TRACE is deliberately built to the ARC-AGI template, and the comparison clarifies both what it borrows and what the formal backbone adds.

What it shares with ARC-AGI. Small, self-contained instances that fit on a screen; a novel "rule of the world" per puzzle so success cannot come from recall; a fixed, simple answer format scored automatically; a few-attempt protocol; difficulty calibrated against human panels; and the target property of being *approachable for people yet hard for machines*—a design goal to be validated by the human pilot (roadmap step V6), not assumed—testing fluid reasoning rather than crystallized knowledge [3].

What the formal backbone adds. Three properties follow from grading against a predicate rather than a stored grid:

(1) **Decidable, generated ground truth.** ARC tasks are hand-authored and their answers hand-drawn. TRACE answers are *computed and checked* against the actual-cause predicate (Section 3.2) from synthesized artifacts—certified against that predicate, and conjectured to equal CORP's labels (Conjecture 1)—so a corpus can be *procedurally generated at scale* with labels correct by construction of the checker and with dialed-in difficulty.

(2) **Set-valued correctness.** ARC accepts one output grid. Temporal answers can be genuinely non-unique—several actual causes in *Credit* (Puzzles 3–5), pivotal switches in *Prevent* (Puzzle 4), or determining sets in *Pin* (Puzzle 3)—and the native grader accepts *any* correct minimal witness, a more faithful notion of "correct reasoning" than exact match, available only because the predicate is decidable.

(3) **A built-in experiment affordance.** The probe sandbox gives a principled, budgetable notion of "testing a hypothesis" that ARC's static grids do not, and that cleanly separates pure reasoning ($\beta_{\max} = 0$) from guided experimentation.

What is different in kind. The analogy is to *format and rigor*, not to the cognitive loop. ARC tests *inductive* spatial abstraction over *Core Knowledge* priors (objectness, geometry, counting) [3]: infer a static rule from examples. TRACE tests *deductive/abductive* temporal credit assignment—simulate or probe a hidden machine over time and reason counterfactually about which inputs decided an event. These are complementary axes of fluid intelligence; an agent strong on one need not be strong on the other, and we borrow ARC's packaging without claiming it measures the same thing.

	ARC-AGI	**Trace**
Instance	input/output grid pairs	reactive device + switch/light run + target flash
Reasoning axis	spatial abstraction	temporal actual causation (necessity + sufficiency)
Answer	an output grid	a set of timestep-indexed switches
Ground truth	hand-authored	computed; checked vs. predicate ($\approx$ CORP, Conj. 1)
Uniqueness	single grid	set-valued (any minimal cause)
Generation	human design	procedural from synthesized artifacts
Grading	exact-grid match	actual-cause / validity + minimality predicate
Experiment	none	budgeted counterfactual probe
Priors tested	Core Knowledge	machine simulation + intervention
Calibration	timed human panels	timed human panels (same recipe)

7 Roadmap: From Design to Benchmark (TODO)

The formal design is complete; what follows is the ordered list of validation and build steps that turn it into a released benchmark. Each step is stated as a falsifiable TODO with its success criterion; steps V1–V2 are the gatekeepers for every TempoBench-equivalence claim in this paper.

V1. Artifact audit ((F1)–(F3)). Parse the released TempoBench HOA artifacts and check, per file: that it is an implementation rather than a specification monitor (F1), input-determinism (F2), and input-totality (F3). Report the fraction of the corpus satisfying each. *Success:* a quantified scope for Proposition 2 and Conjecture 1; failures filtered or the claims narrowed accordingly.

V2. CORP correspondence (Conjecture 1). Run TempoBench's key generation (its Algorithm 1, which invokes `corp` [11, 8]) on a representative sample of (F1)–(F3) instances and compare its per-cell keys with Γ^{ac}. Also pin down, from [5], whether the underlying contingency discipline permits non-actual values, as Definition 4 does. *Success:* per-cell agreement rate with disagreements published as failure cases; on disagreement, either the definition or the alignment claim is revised.

V3. Grader implementation and profiling. Implement the checker of Appendix B; measure runtime versus $|I|$ and k for each mode; publish the tractable $(|I|, k)$ envelope that the verifiable-reward claim of Section 5 is then restricted to. Include at least one fully worked HOA-derived instance (replacing reliance on the hand-built puzzles of Section 4.6 as the only evidence the definitions survive contact with real artifacts).

V4. Generator and corpus statistics. Build the generator (P1–P6), produce a prototype corpus, and report: instance counts per difficulty stratum; the frequency of the no-pivotal-cell verdict, split into overdetermined versus input-unconditional instances; the per-cell agreement rate of *Credit* and *Pin* (Section 2.6)—a per-cell disagreement instance, if found, is the example that justifies *Credit* specifically; and the trivial-instance discard rate.

V5. Black-box identifiability enforcement. Implement the admission check of Section 5: an instance enters the black-box track only if all devices consistent with the observed run and budget-reachable probe information agree on the native answer. Report how restrictive the check is in practice.

V6. Human pilot and calibration. Following ARC-AGI-2 [4]: recruit a panel, present puzzles under a time limit and the standard attempt budget, retain an evaluation instance only if reliably solved by multiple participants, and report per-stratum solve rates against the difficulty knobs and the active mode. This is the test of the "approachable for humans" hypothesis, including for *Credit*, whose contingency search may be the hardest mode for non-experts.

V7. Agent baselines. Evaluate, in both tracks: prompted models with the probe tool; prompted models in the pure-reasoning regime ($\beta_{\max} = 0$, white-box); reasoning-trained models; and the generator's own search as a non-learning ceiling. Report native, TS, and AP scores per mode, probe usage, and the no-pivotal-cell recognition rate. Primary metric: native solve rate; the human–agent gap per stratum is the quantity the benchmark exists to track.

V8. Release. Corpus, checker, canonical keys, and the JSON protocol of Appendix B, versioned so that V1–V7 results are reproducible.

What success would show. A useful benchmark exhibits a wide human–agent gap that narrows only with genuine improvements in temporal and counterfactual reasoning, and resists brute-force probing (hence the budget). If agents close the gap by probing exhaustively, tighten $\beta_{\max}$; if they close it by a heuristic, the non-monotonicity result (Remark 2) says a heuristic cannot be exactly correct in general, so discriminating instances can be surfaced.

8 Limitations

- **Conditional alignment.** Every TempoBench-equivalence claim rests on (F1)–(F3) and Conjecture 1, to be discharged by roadmap steps V1–V2. If either fails, the grader still defines a clean, decidable game over whatever A_{HOA} *is*, but the claim of TempoBench equivalence lapses and TRACE stands as an independent trellis-causation benchmark.
- **Generation cost.** Computing the actual-cause key compounds a minimal-cause search with an outer contingency search; each underlying simulation is polynomial in the explicit (expanded) automaton and k, but the searches are exponential in $|I| \cdot k$ in the worst case (Section 3.2). The difficulty knobs keep production tractable but cap instance size.
- **The simplest mode is degenerate by design.** Count-only *Prevent* collapses on input-deterministic, input-total controllers (Proposition 2); we retain it only as a strict diagnostic, and the core is actual causation. A reader expecting necessity to be the TCE object should note this is exactly why it is not.
- **A genuine ceiling: one-player reach.** The frozen device is adequate for trace-level point-effect causes, which is the whole of TCE, but *cannot* express liveness or strategy-level causes ("was this standing controller commitment necessary against every environment"); those require the two-player reverse parity game of Appendix A, where polynomial-time *generation* is lost.
- **Black-box scores bundle identification with reasoning.** This is deliberate—it is what makes the probe budget meaningful—and it is why the black-box track carries the identifi-

ability requirement of Section 5 (enforced by roadmap step V5) and why white-box is the primary reasoning track.

- **Closeness is a declared choice.** Count-only closeness is canonical but not forced; profile refinement (Appendix A) can change which removals are "closest" and hence, at the margin, validity. A corpus must declare its order.

9 Conclusion

We turned a formal, frame-relative notion of temporal cause into a benchmark. Specializing to point effects under input-only counterfactuals re-grounds the TempoBench TCE task as a self-contained, finite construction on three decidable causal objects—necessity, sufficiency, and actual causation—of which actual causation is the one we conjecture matches TempoBench's CORP-generated labels (a correspondence we state as checkable, not proved, having shown that the simpler count-only necessity provably collapses to singleton causes or the no-pivotal-cell verdict on input-deterministic, input-total controllers). On that backbone we built TRACE: freezing a synthesized controller into a one-player reactive device, played as a Turing-Tumble-style switch-and-light puzzle in three modes—*Credit* (mark the switches that decided the flash; actual cause), *Prevent* (necessity), and *Pin* (sufficiency)—with a budgeted counterfactual probe and a predicate grader that accepts any correct minimal cause. Human and agent are graded on identical information through a JSON protocol with a verifiable reward, in a white-box primary track and a black-box discovery track. The result is a temporal-causal analog of ARC-AGI that adds three things the formal account makes possible: decidable, procedurally-generable ground truth; set-valued correctness; and a principled experiment affordance. The constraints that pin the game to TempoBench are one corner of a lattice the appendix maps. What remains is the roadmap of Section 7: audit the artifacts, check the CORP correspondence, build and profile the generator and grader, and measure the human–agent gap.

A Relaxing the Constraints: The Flexibility Behind the Game

$\mathfrak{F}_{\mathrm{TB}}$ pins each axis of the framework to a simple setting—a point effect, an input-only intervention, a literal vocabulary, an empty background, a finite horizon. The framework is a lattice over those axes, and relaxing any one yields a richer, well-defined game variant that uses the *same* board, move, probe, and validity-and-minimality grading. This appendix walks the axes, says what each relaxation buys, and—because a benchmark needs a grader—flags where automatic grading stays cheap and where it does not.

Throughout, a relaxation is "cheap" if validity for a *fixed* candidate stays a polynomial check in the explicit automaton—as it does for the necessity and sufficiency predicates (Section 3.2)—and "costly" if it pushes the construction to a parity solve, an exponential search, or an undecidable open-vocabulary regime. (The core *Credit* predicate already carries the contingency search of Definition 4, exponential in $|I| \cdot k$, on top of any axis; "cheap"/"costly" here refers to the *axis*, not to that fixed Credit overhead.)

A.1 Axis 1: the effect class

$\mathfrak{F}_{\mathrm{TB}}$ fixes $E_{\mathfrak{F}} = X^k o$: one positive output, one fixed timestep. The effect is, in general, "whatever temporal fact the game asks the player to explain," and it can be enriched along an informal

hierarchy of increasing richness:

$$\text{point output} \rightsquigarrow \text{negative/non-event} \rightsquigarrow \text{bounded temporal} \rightsquigarrow \text{state reachability} \rightsquigarrow \text{liveness} \rightsquigarrow \text{parity acceptance.}$$

Non-event $X^k \neg o$

"why did o *not* fire at k?" A new level, but a subtle one: many facts can independently prevent an output, so non-event causes are often disjunctive, and instances tend to land in the overdetermined regime of Puzzle 3. *Cheap* to check, but expect $\mathsf{Causes} = \emptyset$ in the literal vocabulary more often.

Bounded temporal $F^{\leq w} o$ or $req \to F^{\leq w} grant$ within the window

"which inputs made the response happen within w steps?" Still finite-horizon and *cheap*: build a small bounded monitor and check on the window. This is the natural "level 2" of TRACE: a request/response game rather than a single-cell game.

State reachability $F(at\_s)$

requires exposing internal state as observable atoms (at_s); then "which inputs drove the machine into state s?" *Cheap* once states are exposed, and it shifts the question from outputs to internal dynamics.

Liveness $GF\, o$ and parity acceptance

"does the system grant *infinitely often*?" These are not finite-horizon facts, and the finite-window trace-level checker does not handle them: a recurrence commitment is not removed by any finite set of input-cell edits within a bounded window, so the finite closest-removal test of Definition 2 does not apply (the co-safety scope boundary). Capturing them requires moving to lasso/infinite-trace semantics or, more naturally, to the strategy level (Axis 2), where the question becomes "can the controller drop this commitment and still guarantee the objective?" *Costly*: the grader becomes a parity solve.

A.2 Axis 2: the intervention regime

This is the deepest axis. $\mathfrak{F}_{\mathrm{TB}}$ is *input-only*: the player edits inputs, and the machine is otherwise fixed. Two relaxations open up.

Output-intervention (event causality). Allow the probe to edit *outputs* away from what the machine produces. The question shifts from "which inputs caused o" to "which earlier *events* (including the system's own outputs) caused o"—event-to-event causality rather than input-to-output. Still a one-player, finite check on the window, so *cheap*, but it changes the meaning of a cause and the admissible-counterfactual set.

Strategy-structure (a genuinely two-player game). Here the relaxation is qualitative. Instead of a frozen machine, expose the underlying synthesis game G_φ and let the player *re-open the controller's commitments* on a declared set of vertices $V_{\mathfrak{F}}$: the candidate "cause" is no longer a trace input but a *controller commitment* (e.g. the recurrence $GF\, at\_serve$). The question becomes strategy-guarantee necessity (N2): *can the re-opened controller drop commitment C and still guarantee the effect against every admissible environment?* This is decided not by a one-player edit search but by a *reverse parity game*—solve the parity game for the $\exists$-winning region $W_\exists$ of the

assume–guarantee objective $B_{\mathfrak{F}} \to E$, then a one-player non-emptiness check for $B_{\mathfrak{F}} \wedge E \wedge \neg C$ on the sub-arena $\mathcal{A}[W_\exists]$. As a TRACE variant this is a two-player level: one side proposes a commitment to drop, the other (the environment) tries to break the guarantee. It is exactly the right setting for the liveness and parity effects Axis 1 could not reach. Grading is *costly*: per-candidate validity is a quasi-polynomial parity solve [2] plus a polynomial non-emptiness check (still automatic), but *minimizing* the re-opened set $V_{\mathfrak{F}}$ is a separate subset search with the parity decision as an oracle—no longer a single solve. This variant is what lets TRACE ask the genuinely temporal questions ("which standing commitment of the controller was necessary?") that the point-effect core deliberately excludes.

A.3 Axis 3: the closeness order

$\mathfrak{F}_{\mathrm{TB}}$ uses *count-only* closeness, so "closest" is a *set* of minimum-change removals and validity quantifies over all of them. A frame may instead declare a total tie-break (a *profile refinement*: order the changed cells, then their new values), yielding a *unique* closest removal. The trade-off is real and worth surfacing as a difficulty/where-the-credit-goes knob:

- count-only is robust—it does not let an arbitrary tie-break decide which same-cost intervention "counts"—but admits several closest removals, so a cause must defeat all of them;
- profile refinement gives a single witness (cleaner certificates, unique counterexamples) but makes validity depend on the declared order, so two corpora with different tie-breaks can disagree at the margin.

Either is *cheap*. One may also weight edits (some inputs costlier to flip), giving a metric game where the closest intervention is a shortest weighted path.

A.4 Axis 4: the candidate vocabulary

$\mathfrak{F}_{\mathrm{TB}}$ fixes $\mathcal{K}$ to conjunctions of timestep-indexed input literals. Enlarging it changes both the answers and the game:

Internal-state observables *at_s*
: let causes mention where the machine *was*, not just its inputs—"the grant happened because the machine was in *pending*." Needed for state-reachability effects (Axis 1) and for strategy-level commitments (Axis 2).

Disjunctions / readable LTL
: admit $a \vee b$ (which resolves Puzzle 3's overdetermination into an actual cause) and templates like $G(a \to Fb)$. Now there is no finite literal enumeration; the search becomes a bounded syntax-guided synthesis (CEGIS) over a grammar, with a preference order on *readability* (formula size, then temporal depth, then permissiveness) selecting among valid causes. *Costly* for generation (bounded-size search is exponential in the bound), though per-candidate validity stays a finite check.

Invariants and commitments
: (e.g. $G(req \to F\,grant)$) are only meaningful—and only non-vacuous—under a reopening regime (Axis 2); under input-only they are invariants of the machine and fail non-vacuity by construction.

The vocabulary also sets the answer's *readability*: a richer grammar can return a single explanatory formula where the literal vocabulary would only return a set of cells (or $\emptyset$).

A.5 Axis 5: the background $B_{\mathfrak{F}}$

$\mathfrak{F}_{\mathrm{TB}}$ takes $B_{\mathfrak{F}} = \mathsf{true}$. A non-trivial background moves facts between *cause* and *setting*, exactly as ordinary explanation does (we credit the flipped switch, not the intact wiring). Two uses:

- **Environment assumptions.** Declaring $G(req \to \neg cancel)$ as background removes the counterfactual where req and $cancel$ coincide, so a cause that was $req \wedge \neg cancel$ collapses to req—the no-cancel fact has not vanished, it has moved into the setting. This is a clean knob for tuning what the game treats as "given."
- **Fairness for liveness.** At the strategy level (Axis 2), liveness is only guaranteeable under an environment-fairness assumption such as $GF\, req$; without it the winning region is empty and every candidate is vacuously necessary. The background is what makes those variants well-posed.
- **Held-fixed obligations (a controlled experiment).** When the effect is one conjunct of a multi-part contract, the background can *hold the other obligations fixed*, so the game varies one target while the rest of the specification stays satisfied—closer to a controlled intervention than to free perturbation.

All *cheap* for finite effects (intersect the admissible set with a background monitor); the fairness use rides along with the Axis-2 parity solve.

A.6 Axis 6: effect scope and aggregation

Two smaller axes complete the picture.

- **Scope $S_{\mathfrak{F}}$.** For a conjunctive effect $E = E_1 \wedge \cdots \wedge E_m$, the frame can select one conjunct to explain (and optionally hold the others as background), so a single contract yields many games—"explain the grant obligation," "explain mutual exclusion"—each a separate puzzle.
- **Necessity, sufficiency, and actual cause.** The *Prevent* mode reports *necessary* conditions, which is why a guard returns separate pivotal switches (Puzzle 4, Proposition 2) and an overdetermined target returns $\emptyset$; the *Pin* mode (Definition 3) reports the dual *sufficient* sets. The core *Credit* mode is the *actual cause* (Definition 4)—a Halpern–Pearl object that layers a contingency quantifier on top of necessity. That quantifier makes it heavier than bare necessity to *generate* (an outer contingency search; Section 3.2), but it is the reading we conjecture matches TempoBench's CORP labels, so TRACE promotes it to the core rather than treating it as an extra. The three modes stay separate and labeled, so a corpus never silently conflates "each fact that was needed," "a set that suffices," and "a fact that decided the outcome under some contingency."

A.7 Axis 7: the horizon

$\mathfrak{F}_{\mathrm{TB}}$ is finite-horizon: the effect sits at a fixed timestep and the window is $0, \ldots, k$. Removing this is what forces the jump to the strategy level. Over infinite traces, co-safety effects (a thing eventually happens) still admit finite-change removals and stay at the trace level, but liveness effects (a thing happens *infinitely often*) do not—their removal can require unboundedly many edits, so the finite trace-level test does not apply and the question must be asked of the *controller*, via Axis 2. The horizon axis is therefore not independent: extending it past co-safety effects *is* the move from the one-player game to the two-player game.

A.8 The lattice, at a glance

Axis	$\mathfrak{F}_{\mathrm{TB}}$ (core)	Relaxed variant (grading)
Effect	point $X^k o$	non-event / bounded temporal (cheap); liveness, parity (costly)
Intervention	input-only	output-intervention (cheap); strategy-structure (costly, 2-player)
Closeness	count-only (set)	profile (unique); weighted edits — both cheap
Vocabulary	input literals	state obs. (cheap); disjunctions/LTL (costly gen.)
Background	true	env. assumptions, fairness, held obligations — cheap
Scope/aggreg.	whole effect; actual cause	per-conjunct (cheap); necessity/sufficiency (cheaper diagnostics)
Horizon	finite window	infinite: co-safety (trace) vs. liveness (strategy)

The reading is: the core TRACE game is one corner; "cheap" relaxations grow the game (more effect types, richer counterfactuals, richer answers) while keeping the same automatic native grader; the "costly" relaxations—disjunctive/LTL vocabularies and the strategy-structure regime—are where the game becomes a two-player or synthesis-search problem and where generating certified ground truth, not merely *checking* a submission, becomes the bottleneck. A corpus designer thus has a single dial set: start at the corner for an ARC-clean single-player benchmark, and move outward to trade automatic ground-truth generation for expressive, genuinely temporal causal questions.

B Interface Schema and Checker

B.1 Instance and answer schema

An instance is the tuple $(A_{\mathrm{HOA}}, \mathrm{AP}, \tau_{\leq n}, X^k o)$ with budgets; the JSON encoding of Section 5 is its serialization. Formally a submission is

$$\texttt{ans} = (\Gamma,\ \texttt{no\_pivot}), \qquad \Gamma \subseteq \{0, \ldots, k\} \times \mathrm{Lit}(I),$$

grouped by timestep for TS/AP scoring, with `no_pivot` the explicit no-pivotal-cell answer—"no literal-conjunction cause exists"—for the empty-cause case (Puzzle 3).

B.2 The grader

```
sufficient(Γ):  # Pin (Def. 3)
if not (τ ⊨ C_Γ and o ∈ α_k):  return False   # actuality
return not (exists admissible β ⊨ C_Γ with o ∉ β_k)  # reachability

valid(Γ):  # Prevent (count-only necessity, Def. 2)
if not (τ ⊨ C_Γ and o ∈ α_k):  return False   # actuality
R := closest_removals(Γ)   # min input-cell edits making an admissible β ⊭ C_Γ
if R is empty:  return False   # non-vacuity
return all(o ∉ β_k for β in R)   # counterfactual dependence

actual_cause(Γ):  # Credit:  AC1--AC2 of Def. 4; AC3 (minimality) is score_native's
job
if not (τ ⊨ C_Γ and o ∈ α_k):  return False   # AC1
for each contingency (W, w), W ⊆ other in-window cells, w an assignment (possibly
non-actual):
if o ∉ β⟨W=w, Γ=act⟩_k   # AC2(a):  removing Γ kills o
and all(o ∈ β⟨(W\W')=w, W'=act, Γ=act⟩_k for W' ⊆ W):
# AC2(b):  hold W\W' at w, reset W' to actual, for every W' ⊆ W
```

```
return True
return False

closest_removals(Γ):
BFS over # changed input cells in window, on product of A_HOA with a
monitor enforcing ¬C_Γ + tracking o@k; return all min-cost admissible β.

score_native(Γ, mode):
pred := {credit: actual_cause, prevent: valid, pin: sufficient}[mode]
if mode = prevent and no_pivot-claim: # instance-level certificate, not one call
return (no singleton {c} has valid({c}))
return pred(Γ) and (no Γ′ ⊊ Γ has pred(Γ′)) # correct & subset-minimal
```

Costs are mode-dependent (Section 3.2): `sufficient` is a reachability/emptiness query, polynomial in the explicit $|A_{\mathrm{HOA}}|$ and k; `valid` (closest-removal necessity) is a shortest-path search over the explicit product, also polynomial in $|A_{\mathrm{HOA}}|$, k, and $|I|$; `actual_cause` adds an outer search over $\leq 3^{|I|(k+1)}$ partial contingency assignments—each binary in-window cell unconstrained, held actual, or held flipped (Section 3.2)—with up to $2^{|W|}$ subset-reset checks each, and is therefore exponential in $|I| \cdot k$. `score_native` adds $2^{|\Gamma|}$ predicate calls for minimality, small because minimal causes are small. The *no-pivotal-cell* verdict is an instance-level certificate, *not* a single `valid` call: on input-total A_{HOA} it reduces to checking the $O(|I|(k+1))$ single flips (Proposition 2), and otherwise to a bounded search, so we report its cost separately. TS and AP scoring compare Γ to a shipped canonical $\Gamma^\star$ per timestep and per cell respectively.

C Notation

A_{HOA}	supplied finite acceptor over $2^{I \cup O}$ (Definition 1); $\mathcal{L}(A_{\mathrm{HOA}})$ is the system's behavior set under (F1).
$\tau_{\leq n}$	finite observed trace $\alpha_0 \cdots \alpha_n$, $\alpha_t \subseteq I \cup O$.
$E_{\mathfrak{F}} = X^k o$	point effect: output o at timestep k.
$\mathfrak{F}_{\mathrm{TB}}$	the TempoBench frame: input-only, $B_{\mathfrak{F}} = \mathsf{true}$, literal vocabulary, count-only closeness, subset minimality.
Γ, C_Γ	constraint set $\subseteq \{0, \ldots, k\} \times \mathrm{Lit}(I)$ and its formula $\bigwedge X^t \ell$.
$\mathrm{Close}^{\leq k}$	closest (minimum input-cell-change) admissible removals of a candidate.
$\mathsf{Causes}^{\leq n}$	count-only *necessity* causes (Definition 2, finite form; the *Prevent* mode); $\Gamma^\star$ a subset-minimal one; $\mathsf{Causes}^{\leq n} = \emptyset$ is the *no-pivotal-cell* verdict (Proposition 2).
determining set	subset-minimal *sufficient* Γ (Definition 3; the *Pin* mode).
Γ^{ac}	the *actual-cause* set (Definition 4; the core *Credit* mode): cells that are counterfactually necessary under some contingency.
CORP	the cause-synthesis tool TempoBench uses to label TCE [8, 11]; the per-cell key derived from it is conjectured to equal Γ^{ac} (Conjecture 1).

(F1)/(F2)/(F3) behavior-model / functionality / input-totality assumptions on A_{HOA} (Remark 1, Assumption 1).

$\beta_{\max}$ probe budget; $\beta_{\max} = 0$ is the pure-reasoning (white-box) regime.

N1 / N2 trace-existential (one-player) / strategy-guarantee (two-player) necessity (Appendix A.2).

References

[1] Tomáš Babiak, František Blahoudek, Alexandre Duret-Lutz, Joachim Klein, Jan Křetínský, David Müller, David Parker, and Jan Strejček. The Hanoi omega-automata format. In *Computer Aided Verification (CAV 2015)*, volume 9206 of *Lecture Notes in Computer Science*, pages 479–486. Springer, 2015.

[2] Cristian S. Calude, Sanjay Jain, Bakhadyr Khoussainov, Wei Li, and Frank Stephan. Deciding parity games in quasipolynomial time. In *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing (STOC 2017)*, pages 252–263, 2017.

[3] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

[4] François Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A new challenge for frontier AI reasoning systems. *arXiv preprint arXiv:2505.11831*, 2025. v1 May 2025; v2 January 2026.

[5] Norine Coenen, Bernd Finkbeiner, Hadar Frenkel, Christopher Hahn, Niklas Metzger, and Julian Siber. Temporal causality in reactive systems. In *Automated Technology for Verification and Analysis (ATVA 2022)*, Lecture Notes in Computer Science. Springer, 2022.

[6] Giuseppe De Giacomo and Moshe Y. Vardi. Linear temporal logic and linear dynamic logic on finite traces. In *Proceedings of the 23rd International Joint Conference on Artificial Intelligence (IJCAI 2013)*, pages 854–860, 2013.

[7] Luca Di Stefano. Execution and monitoring of HOA automata with HOAX. *arXiv preprint arXiv:2507.11126*, 2025.

[8] Bernd Finkbeiner, Hadar Frenkel, Niklas Metzger, and Julian Siber. Synthesis of temporal causality. In *Computer Aided Verification (CAV 2024)*, Lecture Notes in Computer Science. Springer, 2024. arXiv:2405.10912.

[9] Joseph Y. Halpern. A modification of the Halpern–Pearl definition of causality. In *Proceedings of the 24th International Joint Conference on Artificial Intelligence (IJCAI 2015)*, pages 3022–3033, 2015. arXiv:1505.00162.

[10] Joseph Y. Halpern and Judea Pearl. Causes and explanations: A structural-model approach. Part I: Causes. *The British Journal for the Philosophy of Science*, 56(4):843–887, 2005.

[11] Nikolaus Holzer, William Fishell, Baishakhi Ray, and Mark Santolucito. Mechanics of learned reasoning 1: TempoBench, a benchmark for interpretable deconstruction of reasoning system performance. *arXiv preprint arXiv:2510.27544*, 2025. Implementation: https://github.com/nik-hz/tempobench.

[12] Swen Jacobs, Felix Klein, and Sebastian Schirmer. A high-level LTL synthesis format: TLSF v1.1. In *Proceedings of the Fifth Workshop on Synthesis (SYNT@CAV 2016)*, 2016. arXiv:1604.02284.

[13] Swen Jacobs, Guillermo A. Pérez, et al. The reactive synthesis competition (SYNTCOMP): 2018–2021. *International Journal on Software Tools for Technology Transfer*, 2024. arXiv:2206.00251. TODO: expand the full author list from the published version before camera-ready.

[14] Florian Renkin, Philipp Schlehuber-Caissier, Alexandre Duret-Lutz, and Adrien Pommellet. Dissecting ltlsynt. *Formal Methods in System Design*, 2023.